Centro Universitário de Excelência
Sistemas de Informação

MecaniQA API — OAT 3. Persistência com JDBC e Camada de Serviço

Autor:
Arthur Machado Morais
Felipe Souza Teixeira da Silva
Ian da Silva Borges
Jonatan de Souza Ferreira
Victor Tadeu Araújo Augusto

Itabuna, 2026

Agenda

O objetivo desta apresentação é demonstrar a evolução da API REST da MecaniQA, aplicando conceitos avançados de Orientação a Objetos como DTOs, Padrão Builder e relacionamentos entre entidades para o gerenciamento completo de ordens de serviço.

1. Motor de Armazenamento:
MySQL e Script SQL versionado.

2. JDBC Puro:
Implementação manual de queries (INSERT, SELECT, UPDATE, DELETE).

3. Mappers de ResultSet:
Conversão de tabelas para Objetos.

4. Camada de Service:
Centralização das regras de negócio.

5. Exceções Customizadas e Tratamento Manual (sem @ExceptionHandler).

6. Demonstração e resultados

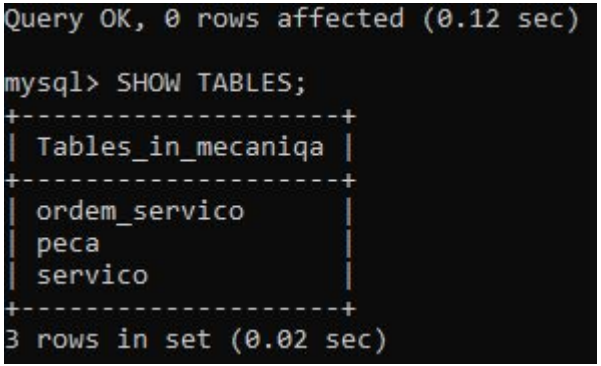
Versionamento e Estrutura do Banco

O sistema agora utiliza o banco de dados MySQL.

O script DDL e DML foi criado e versionado na pasta database/schema.sql do projeto.

Tabelas criadas: peca, servico e ordem_servico.

Uso de chaves primárias auto-incrementais (AUTO_INCREMENT) para geração automática de IDs/códigos.



Fluxo de Dados e Responsabilidades

Controller: Recebe a requisição HTTP e devolve a resposta.

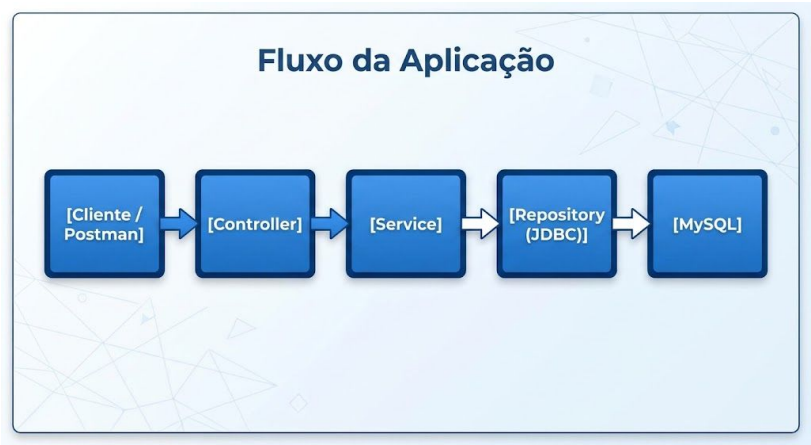
(Não acessa o banco).

Service: Valida as regras de negócio e chama o repositório.

Repository (JDBC): Executa as queries SQL manuais.

Mapper: Transforma o ResultSet em objeto Java.

MySQL: Onde os dados ficam salvos.

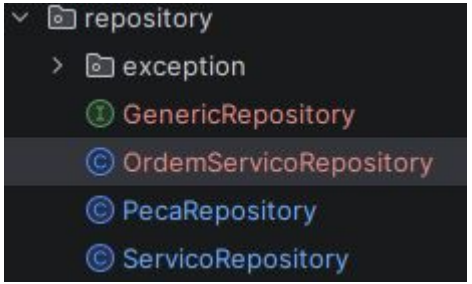


Interface GenericRepository

Para manter o padrão de mercado, criamos nossa própria interface genérica.

Ela define o contrato unificado de CRUD: save, findById, findAll, update, deleteById e existsById.

Benefício: Padronização da arquitetura, mesmo com a implementação JDBC manual.

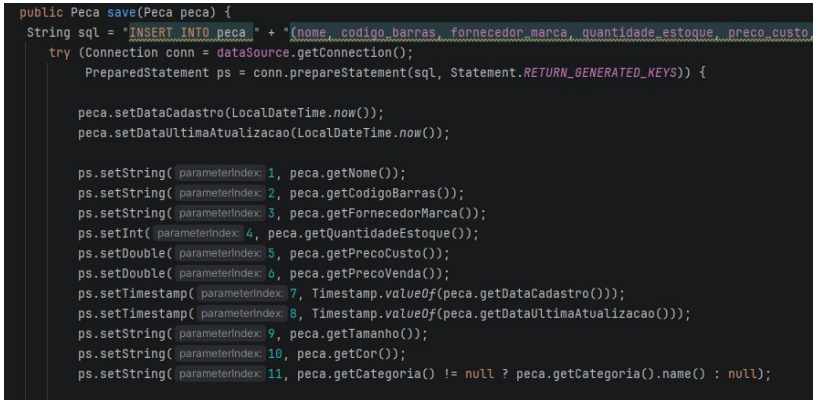



```
@Repository
public class OrdemServicoRepository implements GenericRepository<OrdemServico, Long> {
```

Conexão Manual e Mapeamento de Resultados

Sem ORM (JPA/Hibernate): A conexão é feita via DataSource e as queries SQL são escritas manualmente (PreparedStatement).

Mappers de ResultSet: Como o banco retorna tabelas, criamos métodos (ex: mapResultSetToPeca) para ler os dados de forma segura e montar os objetos do sistema.



Centralizando as Regras de Negócio

Regra de Ouro: Os Controllers estão proibidos de acessar diretamente os repositórios.

A camada de Service (PecaService, ServicoService, OrdemServicoService) recebe os DTOs, valida as regras e orquestra a chamada aos repositórios.

Isso garante a correta separação de responsabilidades (Clean Architecture).

```
public Peca salvar(Peca peca) {  1 usage
    validarDados(peca);
    return repository.save(peca);
}
```

Tratamento de Erros Manual

Sem @ExceptionHandler global: O enunciado proíbe o uso de recursos automatizados.

Criamos exceções próprias

O tratamento é feito com blocos try/catch dentro de cada método do Controller.

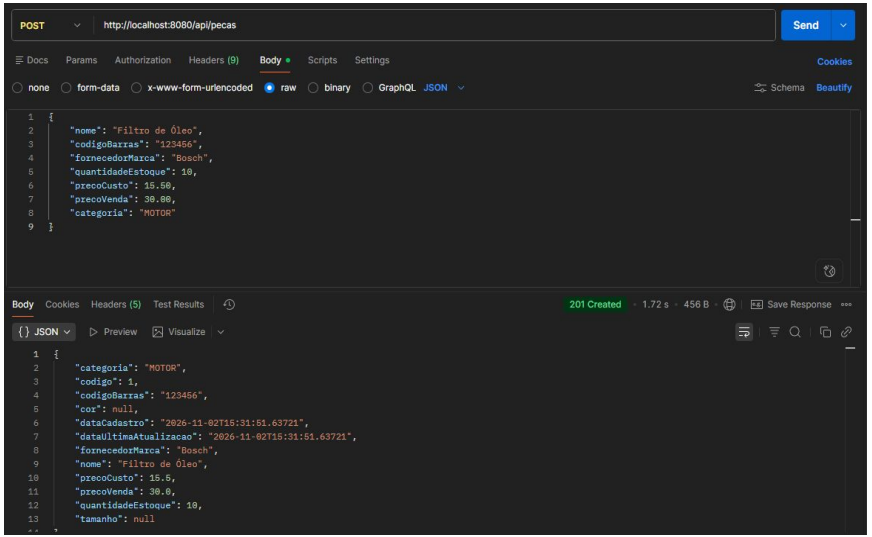
Retornos HTTP controlados manualmente:

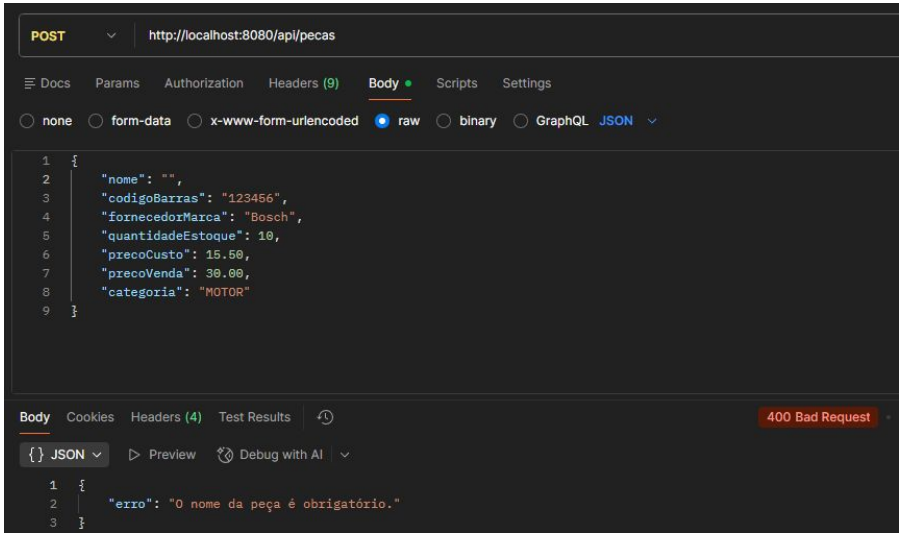
```
package br.com.mecaniQA.api.repository.exception;

public class PecaNaoEncontradaException extends RuntimeException {  10 usages
    public PecaNaoEncontradaException(Long codigo) { super("Peça não encontrada com o código: " + codigo); }
}
```

```
try {
    Peca novaPeca = service.salvar(peca);
    return ResponseEntity.status(HttpStatus.CREATED).body(novaPeca);
} catch (DadosInvalidosException e) {
    return ResponseEntity.badRequest().body(Map.of( k1: "erro", e.getMessage()));
```

Validação e Resultados





Considerações finais

A MecaniQA API agora possui persistência real e robusta.

A implementação manual de JDBC permitiu um entendimento profundo de como a comunicação com o banco de dados funciona por baixo dos panos.

A organização em camadas (Controller -> Service -> Repository -> Banco) prepara o sistema para futuras expansões com segurança.

Referências

- Oracle. MySQL 8.0 Reference Manual. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/

- Oracle. Java SE 21 API Specification (java.sql). Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/module-summary.html

- Spring. Spring Boot Reference Documentation (Data Access with JDBC). Disponível em: https://docs.spring.io/spring-boot/docs/current/reference/html/data.html#data.sql

- Baeldung. Custom Exceptions in Java. Disponível em: https://www.baeldung.com/java-new-custom-exception